



Facultatea
de Automatică
și Calculatoare
Universitatea Tehnică
din Cluj-Napoca



Fundamental Programming Techniques

Assignment 1 - Support Presentation (Part 2)

Computer Science Department, TUCN

Spring 2026



Agenda

- 1 Graphical User Interfaces Development using Swing
- 2 Java Serialization

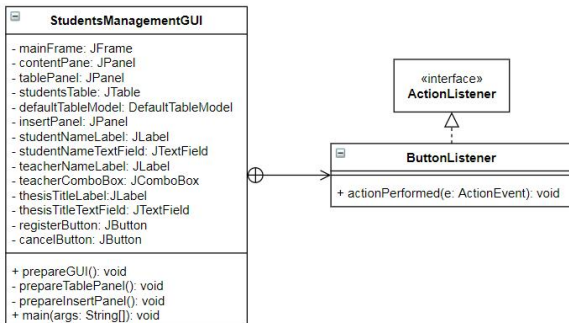


Graphical User Interfaces Development using Swing

- Swing API tutorial
 - ▶ Is part of the Java Foundation Classes (FC)
 - ▶ Offers facilities to write applications with a graphical user interface
 - ▶ Includes 17 packages consisting of classes and interfaces
- ***javax.swing*** is the most important package from Swing

Component Type	Examples
Atomic components	JLabel, JButton, JCheckBox, JRadioButton, JToggleButton, JScrollBar, JSlider
Complex components	JTable, JTree, JComboBox, JList, JFileChooser, JColorChooser, JOptionPane
Text components	JTextField, JPasswordField, JTextArea, JEditorPane, JTextPane
Menus	JMenuBar, JMenu, JPopupMenu, JMenuItem, CheckboxMenuItem, JRadioButtonMenuItem
Intermediate containers	JPanel, JTabbedPane, JDesktopPane
Top level containers	JFrame, JDialog

- Example application: Students Management Application





Graphical User Interfaces using Swing

• Good to know - Top-Level Containers (1)

- ▶ The graphical components must be included in a containment hierarchy having a top level container(e.g. JFrame, JDialog) as root. In particular, the graphical components will be contained in the content pane of the top-level container. A menu bar can be included in a top-level container, but it will reside outside the content pane. To create and set up a frame, the following steps should be performed:
 - ★ Create the frame by instantiating the JFrame class.
 - ★ Create components and add them to the frame's content pane.
 - ★ Size the frame manually (using the setSize() method), or automatically(using the pack method).
 - ★ Show the frame onscreen (using the setVisible())
 - ★ Show the frame onscreen (using the setVisible() method)

Graphical User Interfaces using Swing

• Good to know - Top-Level Containers (2)

- ▶ To get the content pane of a JFrame component, the method `getContentPane()` defined in JFrame class is used. There are 2 approaches for setting the content pane of a JFrame component.
 - 1 Use the method `getContentPane()` defined in JFrame class to get the frame's content pane and add various components to it: `mainframe.getContentPane().add(tablePanel)` . Note, `mainFrame.add(tablePanel)` can also be used as the add method has been overridden and it actually add tablePanel to the frame's content type.
 - 2 Use the JFrame's `setContentPane()` method to make another component the content panel of the frame:

```
1      JPanel contentPanePanel = new Panel();  
2      // add other graphical components to contentPane  
3      mainFrame.setContentPane(contentPanePanel);
```

Graphical User Interfaces Development using Swing

• Example application: Students Management Application

```
private void prepareGUI() {
    mainFrame = new JFrame("Students Management Application");
    mainFrame.setSize(500, 500);
    mainFrame.addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            System.exit(0);
        }
    });
    contentPane = new JPanel(new GridLayout(2, 1));
    prepareTablePanel();
    mainFrame.setContentPane(contentPane);
    mainFrame.setVisible(true);
}
```

The `setVisible(true)` method is used to make the frame appear onscreen.

The `contentPane` `JPanel` component is set as the content pane of the frame. All other graphical components will be contained in it.

The `setSize` method is used to explicitly set the size of the frame.

Adds a window listener to receive window events from the `JFrame` component. In this case, once the frame is closed, the application is exited.

`GridLayout` is used as the *Layout Manager* of the `contentPane` `JPanel` which serves as the content pane of the frame. `GridLayout` organizes the graphical components on rows and columns, setting the same size for all components.

```
private void prepareTablePanel(){
    tablePanel = new JPanel();
    tablePanel.setLayout(new BorderLayout(tablePanel, BorderLayout.PAGE_AXIS));
    defaultTableModel = new DefaultTableModel();
    defaultTableModel.addColumn("Student name");
    defaultTableModel.addColumn("Teacher name");
    defaultTableModel.addColumn("Thesis title");
    defaultTableModel.addColumn(new Object[] {"John Smith", "Jane Black", "E-commerce application using React and Spring"});
    studentsTable = new JTable(defaultTableModel);
    JScrollPane scrollPane = new JScrollPane(studentsTable);
    studentsTable.setFillsViewportHeight(true);
    this.tablePanel.add(scrollPane);
    contentPane.add(tablePanel);
}
```

The `tablePanel` is added to the content pane of the frame.

`BoxLayout` is used as the *Layout Manager* of `tablePanel` in order to organize the contained graphical components on top of each other.

A `DefaultTableModel` object uses a `Vector` of `Vectors` to store the cell value objects of a `JTable` component.

The `JScrollPane` object is used as a container for the `JTable` Component and automatically places the table's header at the top and they remain visible even when the table data is scrolled. If a `JScrollPane` is not used, then the table's header must be manually placed in the container.

Graphical User Interface Development using Swing

Layout Managers are used to organize graphical components in containers. The following Layout Managers can be used [Java UI Swing]

- ➊ **BorderLayout** - places the components in 5 areas: top, bottom, left, right and centre.
- ➋ **BoxLayout** - places the components on a row or on a column.
- ➌ **CardLayout** - enables the implementation of an area that contains different components at different times.
- ➍ **FlowLayout** - places the components in a single row.
- ➎ **GridBagLayout** - places the components in a grid of cells, allowing the spanning and sizing of components over multiple cells.
- ➏ **GridLayout** - sets equal sizes for the components and places them in the requested number of rows and columns.

Java Serialization

- Persist an object state even after the program is not running.
- Object whose class implements the *Serializable* interface. Those objects are serialized into a sequence of bytes that can be written to disk and later restored to recreate the original object.
- Mechanism for implementing a lightweight persistence. The user must explicitly serialize and deserialize the objects in a program.
- A serialized object can be transmitted over the network.
- The transient keyword can be used to turn off serialization for a field.
- Static fields are not serializable.

Java Serialization

```

1 public class SerializationOperations {
2     public static void main(String[] args) throws
        ParseException, IOException,
        ClassNotFoundException {
3         FileOutputStream fileOutputStream = new
            FileOutputStream("john_doe_output.txt");
4         ObjectOutputStream objectOutputStream = new
            ObjectOutputStream(fileOutputStream);
5         objectOutputStream.writeObject(user);
6         objectOutputStream.flush();
7         objectOutputStream.close();
8
9         FileInputStream fileInputStream = new FileInputStream
            ("john_doe_input.txt");
10        ObjectInputStream objectInputStream = new
            ObjectInputStream(fileInputStream);
11        User restoredUser = (User) objectInputStream.
            readObject(); objectInputStream.close();
12        System.out.println(restoredUser.toString());
13    }
14 }

```

```

1 public class User implements
    Serializable {
2     private static final long
        serialVersionUID = 1L;
3     private String username;
4     private transient String
        password;
5     private String name;
6     private LocalDate birthdate;
7     // get and set methods
8 }

```

Used to remember versions of the Serializable classes to verify that a loaded class and the serialized object are compatible.